IMPERIAL

Department of Computing
70015: Mathematics for Machine Learning
Lecturer: Matthew Wicker

Lecture 02: Generalized Linear Models & Vector Calculus

Additional Resources

- Linear Algebra: Chapter 2 (page 29) Bengio et al. (2017) https://www.deeplearningbook.org/contents/linear_algebra.html[1]
- Calculus Review: Chapter 5 (page 139) Deisenroth et al. (2020)
- Linear Regression: Chapter 9 (page 289) Deisenroth et al. (2020) *n.b. This assumes a more probabilistic treatment of linear regression that we will not see until Lecture 5.*

2.1 Ordinary Least Squares

Above we have discussed that if we are able to find the parameter $\theta^\star$ that minimizes our loss, then we have found the best parameter setting for our model. In this section, we will show *how* to find such a parameter in our linear model. Writing out the MSE in summation notation we have that:

$$\mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}}) = \frac{1}{m}\sum_i \left(\boldsymbol{y}_i - \sum_j \boldsymbol{X}_{i,j}\theta_j\right)^2$$

As an exercise in the prior lecture, students were be asked to prove that an equivalent formulation of the above is given by:

$$\mathcal{L}(\theta) = \frac{1}{m}(\boldsymbol{X}\theta - \boldsymbol{y})^\top(\boldsymbol{X}\theta - \boldsymbol{y})$$

Ultimately, we know that maximums and minimums of functions occur when the gradient of the function is equal to zero. As written, the above function is not immediately trivial to differentiate,

[1]Students may also use the following reference for summation notation help: https://www2.ph.ed.ac.uk/~mevans/mp2h/VTF/lecture05.pdf

so we consider algebraic simplifications:

$$\begin{aligned}
&\frac{1}{m}(\boldsymbol{X}\theta - \boldsymbol{y})^\top(\boldsymbol{X}\theta - \boldsymbol{y}) \\
&\quad = \left(\theta^\top(\boldsymbol{X}^\top\boldsymbol{X})\theta - \theta^\top\boldsymbol{X}^\top\boldsymbol{y} - \boldsymbol{y}^\top\boldsymbol{X}\theta + \boldsymbol{y}^\top\boldsymbol{y}\right) \\
&\quad = \left(\theta^\top(\boldsymbol{X}^\top\boldsymbol{X})\theta - 2(\boldsymbol{y}^\top\boldsymbol{X}\theta) + \boldsymbol{y}^\top\boldsymbol{y}\right)
\end{aligned}$$

Now we would like to find where $\nabla_\theta \mathcal{L} = 0$. So, let us start by recalling some vector calculus identities that will help us.

Basic Vector Calculus Rules

For ease of exposition, we are going to break the different rules for vector calculus into *basic* rules and *advanced* rules. The real distinction is that basic rules mirror the single-variable calculus you will have seen in your prior education; all that is needed to derive these rules (or the gradients that only rely on these rules) is the ability to break things down into index notation and apply single-variable calculus.

Each basic rule is the direct analogue of a rule from single-variable calculus, and each can be proven by writing the expression in index notation and differentiating with respect to an arbitrary θ_j. Below, $f, g : \mathbb{R}^n \to \mathbb{R}$, $a \in \mathbb{R}$ and $\mathbf{c} \in \mathbb{R}^n$ are constants, and gradients are row vectors (numerator layout).

Rule	**Single-variable**	**Vector (numerator layout)**
Constant	$\frac{d}{dx}a = 0, \quad \frac{d}{dx}(af) = af'$	$\nabla_\theta a = \mathbf{0}^\top, \quad \nabla_\theta(af) = a\nabla_\theta f, \quad \nabla_\theta \mathbf{c}^\top\theta = \mathbf{c}^\top$
Sum	$(f+g)' = f' + g'$	$\nabla_\theta(f+g) = \nabla_\theta f + \nabla_\theta g$
Product	$(fg)' = f'g + fg'$	$\nabla_\theta(fg) = g\nabla_\theta f + f\nabla_\theta g$
Power	$(x^p)' = px^{p-1}$	$\nabla_\theta f^p = pf^{p-1}\nabla_\theta f, \quad \nabla_\theta(\theta^\top\theta) = 2\theta^\top$

Finally, in what we will consider *basic* vector calculus, we will cover the chain-rule. For a vector-valued function $\mathbf{g} : \mathbb{R}^n \to \mathbb{R}^k$, we write $\nabla_\theta \mathbf{g}$ for its *Jacobian*, the $k \times n$ matrix with entries $(\nabla_\theta \mathbf{g})_{i,j} = \partial g_i / \partial \theta_j$ (numerator layout). When $k = 1$ this is exactly the row-vector gradient we have been using. Then, where $\mathbf{F}$ is a composition of functions $\mathbf{f} : \mathbb{R}^k \to \mathbb{R}^p$ and $\mathbf{g}$, we have:

$$\nabla_\theta \mathbf{F}(\theta) = \nabla_{\mathbf{g}}\mathbf{f}\Big(\mathbf{g}(\theta)\Big) \cdot \nabla_\theta \mathbf{g}(\theta)$$

exactly as in single-variable calculus, except that $\cdot$ is now a $(p \times k)(k \times n)$ matrix product, so the order matters: the outer function is always on the left.

Proving Basic Identities and Deriving the OLS

For vector multiplication we have $\nabla_\theta \mathbf{c}^\top \theta = \mathbf{c}^\top$. Proof:

$$\begin{aligned}\mathbf{c}^\top \theta &= \sum_j \mathbf{c}_j \theta_j \\ \frac{\partial \mathbf{c}^\top \theta}{\partial \theta_j} &= \mathbf{c}_j \\ \nabla_\theta \mathbf{c}^\top \theta &= \mathbf{c}^\top\end{aligned}$$

For quadratic forms, we have:

$$\nabla_\theta (\theta^\top \boldsymbol{A} \theta) = \theta^\top (\boldsymbol{A} + \boldsymbol{A}^\top).$$

Proof:

$$\begin{aligned}\theta^\top \boldsymbol{A} \theta &= \sum_i \sum_j \theta_i \theta_j A_{i,j}, \\ \frac{\partial (\theta^\top \boldsymbol{A} \theta)}{\partial \theta_k} &= \sum_j A_{k,j} \theta_j + \sum_i \theta_i A_{i,k} = (\boldsymbol{A}\theta)_k + (\boldsymbol{A}^\top \theta)_k, \\ \nabla_\theta (\theta^\top \boldsymbol{A} \theta) &= \theta^\top (\boldsymbol{A} + \boldsymbol{A}^\top).\end{aligned}$$

Applying these rules to the form of the loss from we ended up with after distributing we get:

$$\nabla_\theta \mathcal{L} = \frac{2}{m} \Big(\theta^\top (\boldsymbol{X}^\top \boldsymbol{X}) - \boldsymbol{y}^\top \boldsymbol{X} \Big)$$

Now setting it equal to zero and solving we have:

$$\begin{aligned}0 &= \frac{2}{m} \Big(\theta^\top (\boldsymbol{X}^\top \boldsymbol{X}) - \boldsymbol{y}^\top \boldsymbol{X} \Big) && (2.1) \\ &= \theta^\top (\boldsymbol{X}^\top \boldsymbol{X}) - \boldsymbol{y}^\top \boldsymbol{X} && (2.2) \\ \theta^\top (\boldsymbol{X}^\top \boldsymbol{X}) &= \boldsymbol{y}^\top \boldsymbol{X} && (2.3) \\ \theta &= (\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y} && (2.4)\end{aligned}$$

This final equation is known as the *ordinary least squares estimate*. So by selecting θ to be this value, we will have minimized our loss function.

2.2 Basis Expansion

One critical flaw of this model that we have not completely addressed yet is its lack of power. Recall that in childhood we studied models of the for $y = mx + b$ which, without any centering

or normalization are able to fit the data. So how is it that this model we have just presented is less powerful?

Let's formally consider what is seemingly a drawback of the linear transformation requirement. It is easy to see that the final condition in this definition implies the preservation of the origin, that is, $T(\mathbf{0}) = \mathbf{0}$ as we discussed in our last lecture. But if we look at our $f(x) = mx + b$ model for fixed $m, b \in \mathbb{R}$, we have that $f(0x) = b \neq 0f(x)$.

The key to improving the power of our model is to ensure that the model is linear in its parameters, and not linear in the data, necessarily. In this section we will see how to expand our linear model to capture the affine transformation $y = mx + b$.

Affine Basis Expansion This technique is called basis expansion. To recover $y = mx + b$, we simply can add a 1 entry to the end of each feature vector $x \in \mathbb{R}$ and then the corresponding weight for that feature vector becomes the bias. Observe that by introducing a function $\phi(x) : \mathbb{R} \to \mathbb{R}^2$ such that $\phi(x) := [x, 1]^\top$ we then need a parameter vector $\phi \in \mathbb{R}^2$ and so we end up with the model $\hat{y} = \phi(x)^\top \theta = x\theta_1 + 1\theta_2$. Thus θ_2 is effectively our 'y intercept' from our previous model. This can be viewed as the most simplistic form of basis expansion.

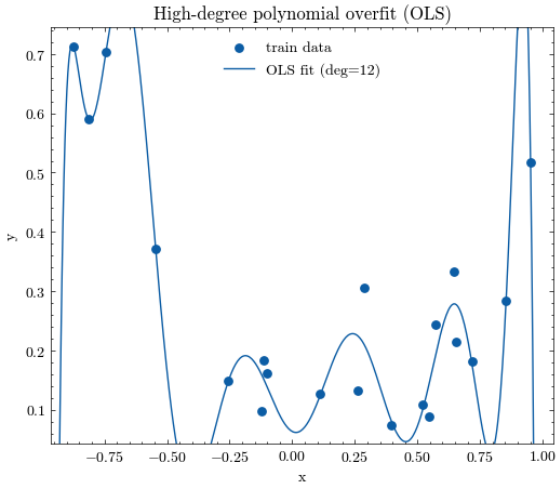
Polynomial Basis Expansion Though our linear model is, as we have formulated it, restricted to lines through the origin, we now present basis expansion as a way of modelling non-linear functions. To begin, lets start with the simplest example, a one dimensional domain and a one dimensional co-domain. The key idea of basis expansion is to (1) expand our one-dimensional feature into many dimensions and (2) use non-linear functions to increase the expressiveness of our previously linear model. We start by considering a basis function $\phi(x) : \mathbb{R} \to \mathbb{R}^3$, specifically we will first consider polynomial basis of the form $\phi(x) = [1, x, x^2]^\top$. We can then assign weights $\theta \in \mathbb{R}^3$ to each of our expanded features. This makes our fully expanded function:

$$\hat{y} = \theta_0\phi(x)_0 + \theta_1\phi(x)_1 + \theta_2\phi(x)_2 \tag{2.5}$$
$$= \theta_0 + \theta_1 x + \theta_2 x^2 \tag{2.6}$$

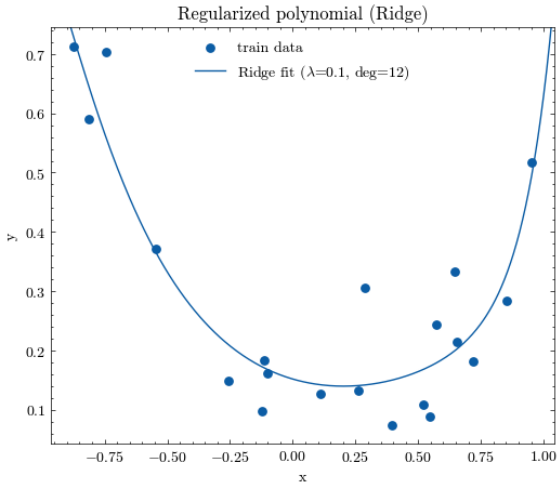
We can see that the output of the function is linearly related to the expanded features, but of course the form of the model that we have expressed is actually a quadratic function. Of course this basis expansion does not just work in one dimension. For example, if our unexpanded feature vector is two dimensional (as opposed to the one dimensional example we just gave), then we can have a basis expansion $\phi : \mathbb{R}^2 \mapsto \mathbb{R}^6$, and following the specific example from before this expansion would return the vector $[1, x_1, x_2, x_1^2, x_2^2, x_1x_2]^\top$. As a minor exercise, you can expand this function as we have done above in the one dimensional case.

Radial Basis Expansion Importantly, polynomial basis expansion is just a single flavor of basis expansion. Another popular and widely used form of basis expansion is kernel basis expansion. Though we do not fully cover the extent of kernel basis expansion models here, they are good to work with to practice key skills (e.g., the vector calculus we will practice at the end of this lecture). A kernel is a function that takes in two points, x and x', and returns a non-negative distance

between those points. The most common kernel is the *radial basis function kernel* (a.k.a. RBF kernel) which takes a fixed parameter γ and is defined as $\kappa(x', x) = \exp(-\gamma||x - x'||^2)$. Typically, one picks fixed centers x and then the basis expansion proceeds by computing the expanded feature set with respect to the distance to these centers.



(a) High-degree polynomial fit (OLS) shows clear overfitting to noise.



(b) Ridge-regularized fit mitigates overfitting by penalizing large coefficients.

Figure 2.1: Comparison of ordinary least squares (OLS) and ridge-regularized polynomial fits. The OLS model (left) chases the noisy data, while the ridge model (right) yields a smoother, more stable solution.

2.3 Regularization

While one can chose different model complexities by choosing different basis expansions, it is clear that this can rapidly lead to models that are needlessly complex (an observation known as overfitting that we will unpack later in the course). A classical way to controlling the complexity of our machine learning model even when the basis expansion enables extremely complex functions is called "regularization."

The general idea of regularization is to introduce a parameter (not necessarily related to the model construction) that limits model complexity. An important aside is that this parameter is considered a *hyper-parameter* which essentially means it is a value that is not the subject of the learning algorithm (e.g., ordinary least squares). The topic of establishing good hyper-parameters will be revisited in later lectures.

One classical example of regression that is usually implemented by default in machine learning frameworks is weight decay. The use of ℓ_2 weight decay in a linear regression loss yields what is known as the ridge regression loss and is given as:

$$\mathcal{L}_{\text{ridge}}(\theta) := \mathcal{L}(\theta) + \lambda||\theta||_2$$

The effect of this term is that parameter or weight values that have high ℓ_2 magnitude also incur high loss. Thus, in our polynomial regression, all coefficients including those on higher-order terms have reduced magnitude. It is intuitively clear that if we set λ to be large, then we are enforcing our parameters to have very low magnitude, tending to 0 magnitude in the limit of λ going to infinity. In your exercises for this lecture you will be given the ridge regression loss for linear regression:

$$\mathcal{L}_{\text{ridge}}(\theta) := \frac{1}{k}||\mathbf{y} - \boldsymbol{X}\theta||^2 + \lambda||\theta||_2^2$$

and asked to solve for the optimal parameter. In this lecture, we look at the practical effects of this regularization.

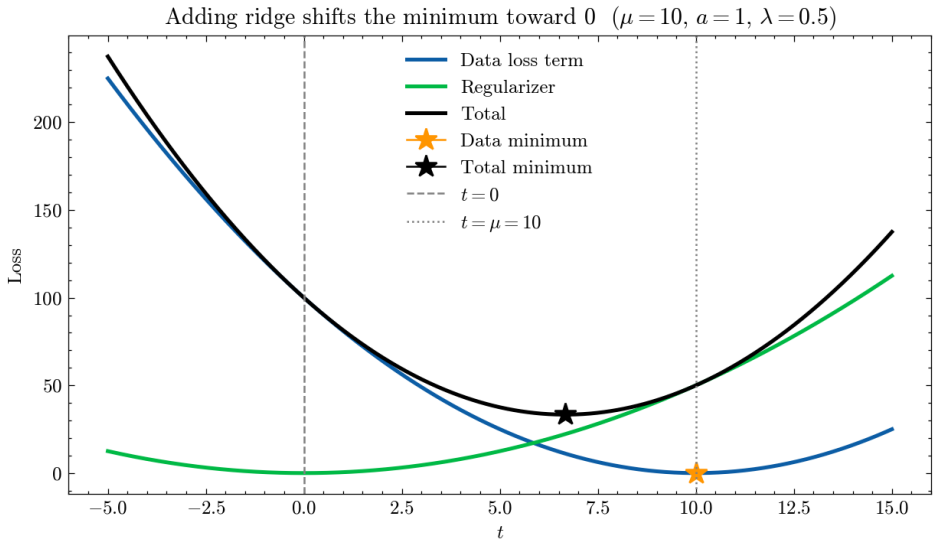


Figure 2.2: Effect of ridge regularization on a simple 1-D quadratic loss. The data-fit term (blue) has its minimum at the true parameter value, while the ridge penalty (green) pulls the total objective (black) toward zero, illustrating how regularization shrinks parameter estimates.

In Figure 2.1 we consider learning a polynomial regression (linear regression with polynomial basis expansion) using a high degree polynomial. Because the degree of the polynomial is so high, we can see in subfigure (a) that the OLS solution is indeed minimizes the error i.e., the distance between the our predictions (blue line) and the data points, but does so in a way that is arguably too chaotic. In subfigure (b) we plot what happens when we introduce some minor regularization ($\lambda = 0.1$) – we now see that the function fits the data, but exhibits much more intuitive smoothness. We will peel back further mathematical motivations for this kind of regularization when we interact with probability. But, for now, it is important to peel back *why* this is happening in our loss landscape.

In Figure 2.2 we plot a simple one dimensional slice of what is happening at the level of our loss landscape. We name the parameter t – because your instructor was short sided while coding, I should have stuck with θ. You can see the blue line is the OLS landscape, with an optimal parameter at 10.0. When we introduce a regularizing function (the green line) we have that the

regularization penalty acting on the point $t = 10.0$ overcomes the fact that it was the minimizer. Now, the minimizer sits somewhere between the regularization minimizer at $t = 0.0$ and the OLS minimizer $t = 10.0$. In fact, we plot this with a black line which is the sum of the two green lines and yields a total minimum around $t = 7.0$ – a much lower magnitude solution!

2.4 Generalized Linear Models (GLMs)

We have seen in our last lecture how we can use basis expansion to expand the power of our linear model. In the most basic example we showed affine basis expansion which allows us to recover the standard form of linear regression we have seen before and more: arbitrarily complex polynomials. Such models are great for modeling variables y vary continuously with respect to either their features or basis-expanded features. Additionally, the linear regression model has several considerable advantages: (1) closed-form solution, (2) efficient inference, and (3) easy to analyze. However, there are settings where linear regression is not suitable, in particular, what if y does not vary continuously with changes to features? What if we have constraints on y?

If linear regression can be an arbitrarily powerful function approximator (formal statement of this still to come), then why shouldn't we just increase model strength until it is suitable for the setting? We have seen overfitting as one answer to this question, however, another answer is that even basis expansion cannot ensure that output conditions are met e.g., that your output is always non-negative. For this, we need a different kind of model.

Generalized linear models use a single non-linear function to encode our beliefs about the output of our model/label generating functions. The general form of the model is:

$$\boldsymbol{y} = g^{-1}(\theta^\top \boldsymbol{x})$$

where g is called the "link" function. Traditionally the link function is associated with a probability density function from the exponential family; however, as in our prior lecture, we will avoid diving into probability theory until later lectures when this interpretation will become very useful.

2.4.1 Poisson Regression

In many applications we are interested in predicting count information. For example, given information about a household, predict how many occupants are in the house. Alternatively, given information about traffic structures predict the number of accidents that will occur in a city per year. It might be intuitive to readers that the previous linear regression models do not restrict their outputs to being strictly positive. In the two applications we just listed before, it makes no sense for negative people to be living in a house hold, nor does it make sense for negative automobile accidents to occur. To tackle these more complicated scenarios we introduce poison regression.

As in our previous regression example we will consider a design matrix $\boldsymbol{X}' \in \mathbb{R}^{m \times n}$ and response variables collected into a response vector $\boldsymbol{y} \in \mathbb{R}^m$. To develop the poison regression model we will first augment $\boldsymbol{X}'$ to $\boldsymbol{X} \in \mathbb{R}^{m \times n+1}$ by adding a 1 to the end of each vector (affine

basis expansion). To achieve the desired strictly positive co-domain we apply perhaps the simplest modification:

$$\hat{\boldsymbol{y}} = \exp(\boldsymbol{X}\theta)$$

However, this simple modification makes it less clear exactly how we quantify our loss function. The loss function we desire here is complex and, without a probabilistic discussion, likely not possible to discover alone:

$$\mathcal{L}(\theta) = \sum_{i=1}^{n} y^{(i)} \boldsymbol{x}^{(i)\top}\theta - \sum_{i=1}^{n} \exp(\boldsymbol{x}^{(i)\top}\theta) - \sum_{i=1}^{n} y^{(i)}! \tag{2.7}$$

We do not have the full context for this equation yet, but we will after we cover probabilistic perspectives on machine learning. One choice (albeit not as principled) we could make is to continue using the MSE loss in which case we can try to proceed as in our ordinary least squares derivation from before:

$$\begin{aligned}
\mathcal{L}(\theta) &= \frac{1}{m}(\hat{\boldsymbol{y}} - \boldsymbol{y})^2 \\
&= \frac{1}{m}(\exp(\boldsymbol{X}\theta) - \boldsymbol{y})^2 \\
&\propto (\exp(\boldsymbol{X}\theta) - \boldsymbol{y})^2 \\
&= (\exp(\boldsymbol{X}\theta) - \boldsymbol{y})^\top(\exp(\boldsymbol{X}\theta) - \boldsymbol{y})
\end{aligned}$$

Now we end up in a position where we can compute the derivative of the above following vector calculus,To compute the gradient, as before, let us first express the loss function in a summation form:

$$\mathcal{L}(\theta) = \sum_{i=1}^{m} \big(\exp(\boldsymbol{X}_{i,:}^\top\theta) - \boldsymbol{y}_i\big)^2$$

where $\boldsymbol{X}_{i,:}$ is the i-th row of $\boldsymbol{X}$, and $\boldsymbol{y}_i$ is the corresponding label. We have written things this way as opposed to $\boldsymbol{x}^{(i)}$ to be more consistent with respect to the linear algebra notation. To compute $\nabla_\theta L(\theta)$, we apply the chain rule. First, differentiate each squared term:

$$\frac{\partial}{\partial\theta}\big(\exp(\boldsymbol{X}_{i,:}^\top\theta) - y_i\big)^2 = 2\big(\exp(\boldsymbol{X}_{i,:}^\top\theta) - y_i\big) \cdot \frac{\partial}{\partial\theta}\big(\exp(\boldsymbol{X}_{i,:}^\top\theta)\big)$$

The derivative of $\exp(\boldsymbol{x}^{(i)\top}\theta)$ with respect to θ is:

$$\frac{\partial}{\partial\theta}\exp(\boldsymbol{X}_{i,:}^\top\theta) = \exp(\boldsymbol{X}_{i,:}^\top\theta) \cdot \boldsymbol{X}_{i,:}^\top$$

Combining our two derivatives according to the chain rule we have:

$$\frac{\partial}{\partial\theta}\big(\exp(\boldsymbol{X}_{i,:}^\top\theta) - y_i\big)^2 = 2\big(\exp(\boldsymbol{X}_{i,:}^\top\theta) - y_i\big) \cdot \exp(\boldsymbol{X}_{i,:}^\top\theta) \cdot \boldsymbol{X}_{i,:}^\top$$

Finally, summing over all k data points, the gradient of $\mathcal{L}(\theta)$ is:

$$\nabla_\theta \mathcal{L}(\theta) = 2\sum_{i=1}^{m}\big(\exp(\boldsymbol{X}_{i,:}^\top\theta) - y_i\big)\cdot\exp(\boldsymbol{X}_{i,:}^\top\theta)\cdot\boldsymbol{X}_{i,:}^\top$$

In matrix form, this can be written as:

$$\nabla_\theta \mathcal{L}(\theta) = 2((\exp(\boldsymbol{X}\theta) - \boldsymbol{y})\circ\exp(\boldsymbol{X}\theta))^\top\boldsymbol{X}$$

Now, setting this equal to zero is where we reach an impasse. Unfortunately, there is no algebraic route forward for isolating θ. This means that introducing even a very simple non-linearity breaks the linear transformation condition and results in the lack of a closed form solution – it just does not exist. Maybe this does not exist because we have not used the proper loss function. To explore this, let's consider another generalized linear model.

2.4.2 Logistic Regression

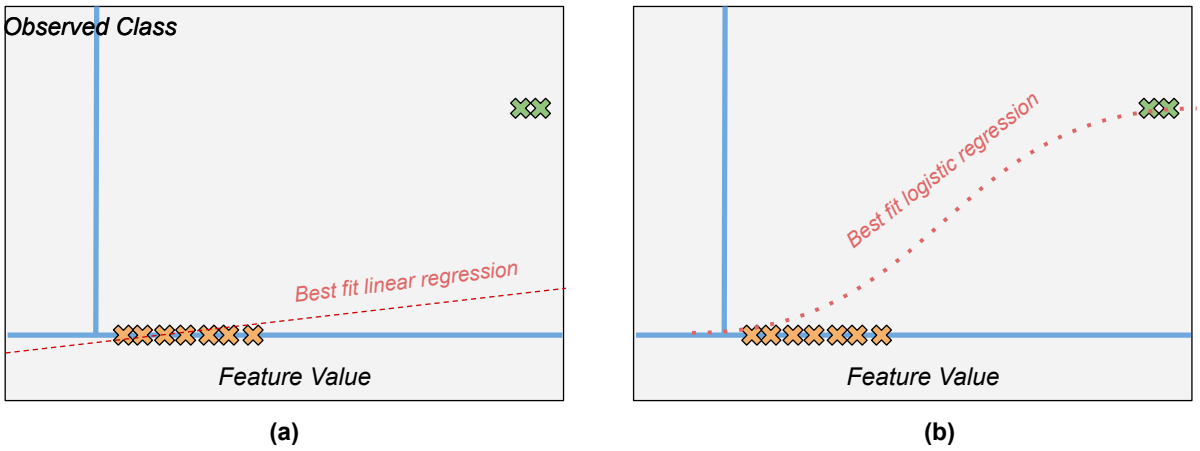


Figure 2.3: Using a contrived, unbalanced dataset we visualize an intuitive case where the non-linearity introduced in logistic regression helps us model the data more accurately.

Another classical example of an application with co-domain restrictions is classification. For example, given information about a patient, predict whether they have a particular disease or given information about a transaction, predict whether it is fraudulent. The simplest case of classification is binary classification: when we only have two classes. In this case, taking $\boldsymbol{y} \in \{0,1\}^N$. Though our final output is in a discrete space, we will restrict our model output to $y \in [0,1]$ and apply a threshold to make the output discrete. We can see that in this case, like the last, we have the non-negative condition *and* the condition that our outputs should not exceed 1. While we will walk through an analogous MSE loss gradient derivation in our lecture slides, we will defer further analysis of this model to the next lecture where we will provide more advanced vector calculus identities and techniques.

Learning Objectives and Techniques

- Vector calculus identities
- Ordinary least squares estimator
- ℓ_p Norms
- Finding extrema of vector-valued functions
- Basis expansion
- Generalized linear models